TASK 3: MUSIC GENERATION WITH AI

Generating New MIDI Music Using an LSTM Neural Network

Project Overview

This project demonstrates AI-based symbolic music generation. It reads MIDI files, converts melodies into note sequences, trains a Long Short-Term Memory (LSTM) neural network to predict the next note, and generates a new sequence that can be saved as a MIDI file.

Tools and Libraries

Python 3 • music21 • TensorFlow/Keras • NumPy • MIDI files

Objective

To learn musical patterns from a collection of MIDI files and use a trained sequence model to create a new melody.

Important

Training quality depends on the number and consistency of MIDI files. The included code is a complete runnable baseline; actual model training and MIDI generation happen when you run it with a local dataset.

1. Problem Statement

Creating music requires learning relationships between notes, timing, and melodic structure. This project uses an LSTM, a recurrent neural network designed for sequence data, to learn note patterns from existing MIDI examples and generate a new note sequence.

2. Workflow

Step	Description
1. Collect data	Place legally obtained or public-domain MIDI files in the dataset folder.
2. Preprocess	Read MIDI files with music21 and extract pitch values; rests are represented with a special token.
3. Build sequences	Create overlapping input sequences and their next-note targets.
4. Train model	Train an embedding + LSTM + dense softmax model to predict the next token.
5. Generate	Seed the model with a note sequence and sample predictions to create a melody.
6. Export	Convert the generated notes into a MIDI file that can be opened in a MIDI player or DAW.

3. Dataset Preparation

Create a folder named **midi_dataset** in the same directory as the Python script and place several .mid or .midi files inside it. Use music you have permission to use. For more reliable results, start with monophonic melodies or MIDI files with a clear lead melody. The script reads all MIDI files in that folder.

4. Install Requirements

```
pip install music21 tensorflow numpy
```

Python 3.10 or 3.11 is a practical choice for matching compatible TensorFlow versions. If installation fails, check the TensorFlow installation instructions for your operating system and Python version.

5. Complete Python Source Code

```
# Task 3: AI Music Generation with an LSTM
# Install: pip install music21 tensorflow numpy
# Add .mid or .midi files to ./midi_dataset/

import os
import glob
import random
import numpy as np
from music21 import converter, note, chord, stream, tempo
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Embedding, Dropout
from tensorflow.keras.utils import to_categorical

DATA_DIR = "midi_dataset"
OUTPUT_MIDI = "generated_music.mid"
SEQ_LENGTH = 40
EPOCHS = 20
BATCH_SIZE = 64

def midi_to_tokens(path):
    """Extract pitch tokens from a MIDI file."""
    try:
        score = converter.parse(path)
        tokens = []
        for element in score.flatten().notes:
            if isinstance(element, note.Note):
                tokens.append(str(element.pitch.midi))
            elif isinstance(element, chord.Chord):
                # Use the chord's pitches as sequential tokens for this baseline.
                tokens.extend(str(p) for p in sorted(element.pitchClasses)
                              if 0 <= p <= 127)
        return tokens
    except Exception as exc:
        print(f"Skipping {path}: {exc}")
        return []

def load_dataset():
    tokens = []
    paths = glob.glob(os.path.join(DATA_DIR, "*.mid"))
    paths += glob.glob(os.path.join(DATA_DIR, "*.midi"))
    for path in paths:
        file_tokens = midi_to_tokens(path)
        if len(file_tokens) > SEQ_LENGTH:
            tokens.extend(file_tokens)
    if len(tokens) <= SEQ_LENGTH:
        raise ValueError(
            f"Not enough note data. Add more MIDI files to '{DATA_DIR}'."
        )
    return tokens

def build_sequences(tokens):
    vocab = sorted(set(tokens))
    token_to_id = {token: i for i, token in enumerate(vocab)}
    inputs, targets = [], []
    for i in range(0, len(tokens) - SEQ_LENGTH):
        inputs.append([token_to_id[t] for t in tokens[i:i + SEQ_LENGTH]])
        targets.append(token_to_id[tokens[i + SEQ_LENGTH]])
    X = np.array(inputs, dtype=np.int32)
    y = to_categorical(targets, num_classes=len(vocab))
    return X, y, vocab, token_to_id

def build_model(vocab_size):
    model = Sequential([
        Embedding(vocab_size, 64, input_length=SEQ_LENGTH),
        LSTM(128, return_sequences=True),
        Dropout(0.2),
        LSTM(128),
        Dense(128, activation="relu"),
        Dense(vocab_size, activation="softmax")
    ])
    model.compile(optimizer="adam",
                  loss="categorical_crossentropy",
                  metrics=["accuracy"])
    return model

def sample_next(probabilities, temperature=0.8):
    # Temperature adjusts randomness: lower = safer, higher = more varied.
    probabilities = np.asarray(probabilities).astype("float64")
    probabilities = np.log(probabilities + 1e-8) / temperature
    probabilities = np.exp(probabilities - np.max(probabilities))
    probabilities = probabilities / probabilities.sum()
    return np.random.choice(len(probabilities), p=probabilities)

def generate_music(model, seed_ids, vocab, id_to_token, count=160):
    pattern = list(seed_ids)
    generated = []
    for _ in range(count):
        x = np.array(pattern[-SEQ_LENGTH:]).reshape(1, SEQ_LENGTH)
        probabilities = model.predict(x, verbose=0)[0]
        next_id = sample_next(probabilities)
```

```
        generated.append(id_to_token[next_id])
        pattern.append(next_id)
    return generated

def save_as_midi(tokens, output_path):
    melody = stream.Stream()
    melody.append(tempo.MetronomeMark(number=110))
    offset = 0.0
    for token in tokens:
        pitch_value = int(token)
        n = note.Note(pitch_value)
        n.quarterLength = 0.5
        n.offset = offset
        melody.insert(offset, n)
        offset += 0.5
    melody.write("midi", fp=output_path)

def main():
    print("Loading MIDI dataset...")
    tokens = load_dataset()
    X, y, vocab, token_to_id = build_sequences(tokens)
    print(f"Training examples: {len(X)} | Vocabulary: {len(vocab)}")

    model = build_model(len(vocab))
    model.summary()
    model.fit(X, y, epochs=EPOCHS, batch_size=BATCH_SIZE,
              validation_split=0.1, verbose=1)
    model.save("music_lstm_model.keras")

    # Start with a random seed sequence from the training examples.
    start = random.randrange(0, len(X))
    seed_ids = X[start].tolist()
    id_to_token = {i: token for token, i in token_to_id.items()}
    new_tokens = generate_music(model, seed_ids, vocab, id_to_token)
    save_as_midi(new_tokens, OUTPUT_MIDI)
    print(f"Done! Generated MIDI saved to: {OUTPUT_MIDI}")

if __name__ == "__main__":
    main()
```

6. How to Run the Project

1. Install Python 3 on your laptop.

2. Create a project folder and save the code as **ai_music_generator.py**.

3. Inside that folder, create a directory named **midi_dataset**.

4. Add multiple MIDI files (.mid or .midi) to midi_dataset.

5. Open a terminal in the project folder and run: `pip install music21 tensorflow numpy`.

6. Run: `python ai_music_generator.py`.

7. After training, open **generated_music.mid** using a MIDI player or music software. The trained model is saved as **music_lstm_model.keras**.

7. Expected Output

```
Loading MIDI dataset...
Training examples: 2500 | Vocabulary: 48
Epoch 1/20
...
Epoch 20/20
Done! Generated MIDI saved to: generated_music.mid
```

The numbers above are illustrative; actual training-example counts, vocabulary size, and training results depend on the MIDI files used. The program creates a MIDI file only after the script runs successfully with a suitable dataset.

8. Result

The project provides a working baseline for AI music generation: MIDI files are converted into pitch-token sequences, an LSTM learns next-note patterns, and the trained model generates a new sequence that is exported to MIDI.

9. Limitations and Future Enhancements

This starter model focuses mainly on pitch sequences and uses a fixed note duration when exporting. It does not fully learn rhythm, velocity, key, harmony, or multiple instrument parts. Future versions can encode pitch plus duration, use separate tracks for polyphonic music, train longer on a larger curated dataset, add a web interface, and export rendered audio using a MIDI synthesizer. Use properly licensed MIDI data and credit sources where required.

10. References

• music21 documentation: https://music21.org/music21docs/
• TensorFlow/Keras documentation: https://www.tensorflow.org/guide/keras
• NumPy documentation: https://numpy.org/doc/